

Fraud Detection Pipeline - Project Report

Project Report: Fraud Detection Pipeline

1. Objective

Build and compare two supervised classifiers — Logistic Regression and Random Forest — to detect fraudulent credit card transactions in a highly imbalanced dataset, with hyperparameters tuned via cross-validated grid search and models evaluated on held-out test data using ROC-AUC and standard classification metrics.

2. Data

- **Source:** `creditcard.csv`, the Kaggle Credit Card Fraud Detection dataset.
- **Rows/Features:** ~284,807 transactions, 30 features (`v1 - v28`, `Time`, `Amount`) plus binary target `Class`.
- **Class balance:** Fraud (`Class == 1`) accounts for roughly 0.17% of transactions — an extreme imbalance that shapes every downstream decision in this pipeline.
- **Preprocessing:**
 - Rows with missing values dropped (`df.dropna()`).
 - `Time` and `Amount` excluded from the feature set — `v1 - v28` (already PCA-transformed and anonymized by the dataset provider) are used directly.
 - 80/20 train/test split, **stratified** on `Class` to preserve the fraud ratio in both splits.

3. Methodology

3.1 Handling Class Imbalance

SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic fraud examples by interpolating between existing minority-class points. It's embedded inside an `imblearn.pipeline.Pipeline` — not applied once to the whole training set — so that during 5-fold cross-validation, resampling is refit on each training fold independently. This avoids a data leakage pattern where synthetic points derived from a validation fold's neighbors would otherwise inflate CV performance.

3.2 Model Pipelines

Logistic Regression pipeline: 1. `StandardScaler` — LR is sensitive to feature scale since its coefficients are penalized/regularized. 2. `SMOTE` — balances classes within the training fold. 3. `LogisticRegression(max_iter=1000)` — increased iteration cap for convergence on the resampled data.

Random Forest pipeline: 1. `SMOTE` — balances classes within the training fold. 2. `RandomForestClassifier` — no scaling needed since tree splits are scale-invariant.

3.3 Hyperparameter Tuning

Both pipelines are tuned with `GridSearchCV`, 5-fold cross-validation, optimizing for **ROC-AUC** rather than accuracy (accuracy is a poor metric here — a model predicting “legitimate” for every transaction would score ~99.8%).

Model	Grid
Logistic Regression	<code>C ∈ {0.1, 1, 10}</code> , <code>solver ∈ {lbfgs, liblinear}</code>
	<code>n_estimators ∈ {100, 300}</code> , <code>max_depth ∈ {5, None}</code> ,

Random Forest	<code>min_samples_split ∈ {2, 5}</code>
---------------	---

3.4 Evaluation

- `classification_report` (precision, recall, F1 per class) on the untouched, original-imbalance test set.
- ROC curve and ROC-AUC, computed from predicted probabilities (`predict_proba`) rather than hard labels, since AUC is threshold-independent.

4. Results

Fill in after running `fraud_detection.py` against the actual dataset — the values below are placeholders.

4.1 Best Hyperparameters & CV Scores

Model	Best Params	Best CV ROC-AUC
Logistic Regression	<code>FILL IN</code>	<code>FILL IN</code>
Random Forest	<code>FILL IN</code>	<code>FILL IN</code>

4.2 Test Set Classification Report

Logistic Regression

`FILL IN` — paste `classification_report(y_test, y_pred1)` output

Random Forest

`FILL IN` — paste `classification_report(y_test, y_pred2)` output

4.3 ROC-AUC (Test Set)

Model	Test ROC-AUC
Logistic Regression	<code>FILL IN</code>
Random Forest	<code>FILL IN</code>

(Insert the two ROC curve plots generated by the script here.)

5. Discussion

- **Precision/recall tradeoff:** In fraud detection, false negatives (missed fraud) are typically far costlier than false positives (flagged legitimate transactions). When comparing models, recall on the fraud class deserves more weight than overall accuracy or even precision alone.
- **SMOTE caveats:** Synthetic oversampling can push the decision boundary in ways that hurt precision, especially for tree ensembles that can already handle imbalance reasonably via class weighting. Worth comparing against `class_weight='balanced'` as an ablation.
- **ROC-AUC vs. PR-AUC:** With ~0.17% positive class prevalence, ROC-AUC can look deceptively strong because the false positive rate axis is dominated by the huge negative class. Precision-Recall AUC is generally considered more informative for this level of imbalance and is a recommended addition.

6. Limitations

- Anonymized `v1 - v28` features limit interpretability — no way to reason about *why* a transaction is flagged without techniques like SHAP.
- Dropping `Time` and `Amount` discards potentially useful signal (e.g., fraud often clusters at specific amounts or times); this was a deliberate simplification, not a proven-optimal choice.
- Single train/test split; no repeated resampling or nested CV to estimate variance in the reported test metrics.

7. Future Work

- Add Precision-Recall curves and PR-AUC alongside ROC-AUC.
- Reintroduce `Amount` (possibly log-transformed) and engineered `Time` features (e.g., hour-of-day) as candidate predictors.
- Compare SMOTE against `class_weight='balanced'` and against gradient-boosted trees (XGBoost/LightGBM).
- Persist the best pipeline with `joblib` and build a minimal inference script/API.
- Cost-sensitive threshold selection instead of the default 0.5 cutoff, based on estimated cost of false negatives vs. false positives.

8. Tech Stack

pandas, numpy, scikit-learn, imbalanced-learn, matplotlib